

Apuntes Profesionales de Aprendizaje Automático

Temas 1, 2, 3, 4, 5, 6 y 9

Preparados a partir de los PDFs de clase

Índice

1. Conceptos Básicos del Aprendizaje Automático	7
1.1. Qué es el aprendizaje automático	7
1.2. Por qué usar aprendizaje automático	7
1.3. Formalización	7
1.4. Tipos de aprendizaje	7
1.5. Supervisado frente a no supervisado	8
1.6. Entrenamiento, test y generalización	8
1.7. Criterios de evaluación	8
1.8. Retos generales	8
1.9. Data-centric AI	8
1.10. Preguntas tipo test del tema 1	9
2. Aprendizaje Supervisado	9
2.1. Idea general	9
2.2. Regresión lineal	9
2.3. Métricas de regresión	9
2.4. Regresión logística	9
2.5. Softmax	10
2.6. k vecinos más cercanos	10
2.7. Naive Bayes	10
2.8. SVM	11
2.8.1. Kernel trick	11
2.9. Árboles de decisión	11
2.10. Ensembles	11
2.11. Clasificación multiclase	12
2.12. Matriz de confusión y métricas	12
2.13. Curvas ROC y PR	12
2.14. Regularización lineal y sesgo-varianza	12
2.15. Regresión polinómica	13
2.16. Descenso de gradiente en modelos lineales	13
2.17. Curvas de aprendizaje	13
2.18. Equilibrio sesgo-varianza	14
2.19. Modelos lineales regularizados en detalle	14
2.20. Parada anticipada o Early Stopping	14
2.21. Regresión logística: pérdida e interpretación	14
2.22. Softmax: entrenamiento multiclase	15
2.23. Compromiso precisión-recall	15

2.24. Notación y normas en regresión	15
2.25. Scikit-Learn: API básica	15
2.26. Aprendizaje basado en instancias frente a modelos	16
2.27. kNN ampliado	16
2.28. Naive Bayes con variables numéricas	16
2.29. SVM ampliado	16
2.30. Árboles de decisión ampliados	17
2.31. Ensembles ampliados	17
2.32. Clasificación multiclase y no balanceada	18
2.33. Data augmentation	18
2.34. Preguntas tipo test del tema 2	18
3. Calidad de Datos y Preprocesamiento	18
3.1. Importancia	18
3.2. Preparación y reducción de datos	18
3.3. Valores perdidos	18
3.4. Ruido y outliers	19
3.5. Normalización	19
3.6. Discretización	19
3.7. Selección de características	20
3.8. Selección de instancias	20
3.9. Reducción de dimensionalidad	20
3.10. Integración de datos	20
3.11. Limpieza y transformación	21
3.12. Imputación avanzada	21
3.13. Ruido en clasificación	21
3.14. Filtros de ruido en detalle	21
3.15. Detección de outliers	22
3.16. Discretización ampliada	22
3.17. Selección de características ampliada	22
3.18. Extracción y construcción de características	22
3.19. Selección de instancias ampliada	23
3.20. Maldición de la dimensionalidad	23
3.21. Proyección, manifolds y PCA	23
3.22. PCA incremental, proyección aleatoria y LLE	23
3.23. Preguntas tipo test del tema 3	24
4. Aprendizaje No Supervisado	24
4.1. Idea general	24

4.2. Clustering	24
4.3. K-Means	24
4.4. Elección de k	24
4.5. Limitaciones de K-Means	25
4.6. DBSCAN	25
4.7. Clustering jerárquico	25
4.8. Otros métodos	25
4.9. Detección de anomalías	25
4.10. Reglas de asociación	26
4.11. Estimación de densidad	26
4.12. Clustering como representación	26
4.13. K-Means ampliado	26
4.14. Métricas de calidad de clustering	27
4.15. DBSCAN ampliado	27
4.16. Clustering aglomerativo ampliado	27
4.17. BIRCH, Mean-Shift, Affinity Propagation y espectral	27
4.18. Retos en detección de anomalías	28
4.19. Métodos de anomalías ampliados	28
4.20. Preguntas tipo test del tema 4	28
5. Aprendizaje Profundo: Fundamentos	28
5.1. Perceptrón	28
5.2. MLP	29
5.3. Funciones de activación	29
5.4. Backpropagation	29
5.5. Batch, mini-batch y época	29
5.6. Inicialización y gradientes	29
5.7. Batch Normalization	30
5.8. Regularización y dropout	30
5.9. Optimizadores	30
5.10. Transfer learning	30
5.11. Salidas según tarea	31
5.12. Contexto histórico y motivación biológica	31
5.13. Perceptrón ampliado	31
5.14. Backpropagation y diferenciación automática	31
5.15. Funciones de activación ampliadas	32
5.16. MLP para regresión y clasificación	32
5.17. Ecosistema y Keras	32
5.18. Entradas, salidas múltiples y multitarea	32

5.19. Ajuste de hiperparámetros	33
5.20. Elección de arquitectura	33
5.21. Tasa de aprendizaje, lote e iteraciones	33
5.22. Problemas de entrenamiento en DNN	33
5.23. Inicialización con fan-in y fan-out	33
5.24. Batch Normalization ampliada	34
5.25. Gradient clipping	34
5.26. Transfer learning, pretraining y autosupervisión	34
5.27. Optimizadores ampliados	34
5.28. Programación de tasa de aprendizaje	34
5.29. Regularización avanzada	35
5.30. Recomendaciones prácticas	35
5.31. Preguntas tipo test del tema 5	35
6. Aprendizaje Profundo en Datos: CNN y Transformers	35
6.1. CNN	35
6.2. Convolución	36
6.3. Ventajas de CNN	36
6.4. Pooling	36
6.5. Tareas de visión	36
6.6. Autoencoders	36
6.7. Secuencias	37
6.8. RNN y LSTM	37
6.9. Atención	37
6.10. Queries, Keys y Values	37
6.11. Self-attention, cross-attention y máscaras	37
6.12. Transformer	37
6.13. Ventajas y limitaciones de Transformers	38
6.14. Representaciones jerárquicas en deep learning	38
6.15. Capa densa frente a capa convolucional	38
6.16. Convolución como operación local	38
6.17. Historia: LeNet	38
6.18. Detalles de convolución	38
6.19. Activaciones, flatten y softmax en CNN	39
6.20. Machine Learning clásico frente a Deep Learning	39
6.21. Denoising autoencoders	39
6.22. Limitaciones de redes densas y CNN en secuencias	39
6.23. RNN y LSTM ampliadas	39
6.24. Atención paso a paso	39

6.25. Multi-head attention	40
6.26. Codificación posicional	40
6.27. Bloque Transformer detallado	40
6.28. Encoder y decoder	40
6.29. Vision Transformers	40
6.30. Transformers en series temporales	40
6.31. Modelos modernos y agentes	41
6.32. Limitaciones adicionales	41
6.33. Preguntas tipo test del tema 6	41
9. Aspectos Avanzados, Retos y Desafíos	41
9.1. Idea general	41
9.2. XAI: IA explicable	41
9.3. Taxonomía XAI	42
9.4. Métodos XAI	42
9.5. Evaluación de explicaciones	42
9.6. Trustworthy AI, Responsible AI y AI Safety	42
9.7. OOD y mundo abierto	42
9.8. Aprendizaje continuo	43
9.9. Machine unlearning	43
9.10. Causalidad y correlaciones espurias	43
9.11. Fairness y sesgos	44
9.12. IA generativa y LLMs	44
9.13. Regulación y gobernanza	44
9.14. RED XAI y BLUE XAI	44
9.15. Taxonomía XAI con ejemplos	44
9.16. SHAP, Integrated Gradients, LRP y CRP ampliados	45
9.17. Prototipos y explicación intrínseca	45
9.18. Evaluación y herramientas XAI	45
9.19. Propuestas XAI avanzadas	45
9.20. Limitaciones abiertas de XAI	45
9.21. Trustworthy AI basada en riesgo	46
9.22. Robustez en el ciclo de vida	46
9.23. Mundo cerrado y mundo abierto	46
9.24. Aprendizaje continuo ampliado	46
9.25. Machine unlearning ampliado	46
9.26. Causal Discovery	47
9.27. Causal Representation Learning	47
9.28. V-REx	47

9.29. Preguntas tipo test del tema 9	48
Chuleta Final de Examen Tipo Test	49

Tema 1. Conceptos Básicos del Aprendizaje Automático

Qué es el aprendizaje automático

El **aprendizaje automático** es el campo que permite a los ordenadores aprender patrones a partir de datos sin programar explícitamente todas las reglas.

Definiciones importantes:

- **Arthur Samuel**: capacidad de aprender sin ser programado explícitamente.
- **Tom Mitchell**: un programa aprende de una experiencia E , respecto a una tarea T y una medida de rendimiento P , si su rendimiento en T , medido por P , mejora con E .

Por qué usar aprendizaje automático

Se usa AA cuando las reglas manuales son demasiado numerosas, rígidas o difíciles de mantener. También permite adaptarse a cambios y descubrir patrones no evidentes en grandes volúmenes de datos.

Minería de datos: aplicación de técnicas de AA para descubrir patrones útiles en grandes cantidades de datos.

Formalización

Un conjunto supervisado se escribe como:

$$\{(x_i, y_i)\}_{i=1}^m$$

donde x_i es la entrada, y_i la etiqueta o salida, y m el número de ejemplos.

El modelo se representa como:

$$f(x, \omega), \quad \omega \in \Omega$$

donde ω son los parámetros y Ω el espacio de modelos posibles.

La función de pérdida L mide el error. El riesgo esperado sería:

$$R(\omega) = \int L(y, f(x, \omega)) p(x, y) dx dy$$

Como $p(x, y)$ no se conoce, se minimiza una aproximación usando los datos de entrenamiento.

Tipos de aprendizaje

Tipo	Datos	Objetivo
Supervisado	Instancias con etiquetas	Predecir clases o valores
No supervisado	Instancias sin etiquetas	Descubrir estructura, grupos o anomalías
Semisupervisado	Pocas etiquetas y muchos datos sin etiquetar	Aprovechar datos no etiquetados
Refuerzo	Agente, acciones y recompensas	Aprender una política que maximice recompensa

Supervisado frente a no supervisado

Clasificación: predice una clase discreta. Ejemplos: spam/no spam, Iris, MNIST, cáncer benigno/maligno.

Regresión: predice un valor numérico. Ejemplos: precio de una vivienda, ventas futuras, temperatura.

Clustering: descubre grupos sin clases conocidas. Ejemplos: segmentación de clientes, agrupación de visitantes de un blog.

Trampa típica: Clasificación y clustering no son lo mismo. En clasificación las clases ya existen y el modelo aprende a asignarlas; en clustering el algoritmo descubre agrupamientos sin etiquetas.

Entrenamiento, test y generalización

Los datos se dividen en entrenamiento y test. El modelo aprende con entrenamiento y se evalúa con datos no vistos.

Error de generalización: error esperado en casos nuevos. También se llama error fuera de muestra.

Una partición habitual es 80 % entrenamiento y 20 % test, aunque con datasets enormes puede bastar un porcentaje menor para test.

Validación cruzada k-fold: divide el conjunto en k partes. Cada vez entrena con $k - 1$ partes y prueba en la restante. Sirve para estimar mejor el rendimiento.

Criterios de evaluación

No todo es accuracy. Un modelo también se evalúa por:

- Coste de clasificar mal.
- Velocidad de entrenamiento e inferencia.
- Robustez ante valores desconocidos, ruido o cambios.
- Escalabilidad con el tamaño de la base de datos.
- Interpretabilidad y complejidad del modelo.

Retos generales

No Free Lunch: no existe un algoritmo universalmente mejor para todos los problemas. Sin supuestos sobre los datos no hay razón para preferir un modelo a otro.

Sobreajuste: el modelo funciona bien en entrenamiento pero mal en datos nuevos.

Regularización: restringir o simplificar el modelo para reducir sobreajuste.

Sesgo de muestreo: los datos de entrenamiento no representan bien los casos reales.

Desajuste de datos: train, validación, test o producción tienen distribuciones distintas.

Data-centric AI

Model-centric AI: intenta mejorar el rendimiento cambiando arquitectura, algoritmo o hiperparámetros, manteniendo los datos casi fijos.

Data-centric AI: mejora sistemáticamente los datos: adquisición, limpieza, etiquetado, auditoría, balanceo y generación.

Preguntas tipo test del tema 1

1. Si un sistema aprende con correos etiquetados spam/no spam, es aprendizaje supervisado.
2. Si un sistema agrupa clientes sin etiquetas previas, es aprendizaje no supervisado.
3. Si el modelo se ajusta usando el conjunto de test, la evaluación queda contaminada.
4. No Free Lunch implica que hay que probar modelos razonables según el problema, no buscar el algoritmo perfecto universal.
5. En aprendizaje por refuerzo, la política decide qué acción tomar en cada estado.

Tema 2. Aprendizaje Supervisado

Idea general

El aprendizaje supervisado aprende una función que relaciona entradas X con etiquetas y . Si y es numérica hablamos de regresión; si y es categórica, de clasificación.

Regresión lineal

Modelo:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \theta^T x$$

Función de coste típica:

$$\text{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2$$

Solución por ecuación normal:

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

si la inversa existe. En la práctica puede usarse pseudoinversa/SVD o descenso de gradiente.

Métricas de regresión

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}$$

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{y}^{(i)} - y^{(i)}|$$

- RMSE penaliza más los errores grandes, porque eleva al cuadrado.
- MAE es más robusta frente a outliers.
- MSE y RMSE tienen el mismo óptimo, porque la raíz es monótona.

Regresión logística

Aunque se llama regresión, se usa para clasificación binaria.

$$p = h_{\theta}(x) = \sigma(\theta^T x), \quad \sigma(t) = \frac{1}{1 + e^{-t}}$$

Con umbral 0.5:

$$\hat{y} = 1 \text{ si } p \geq 0,5, \quad \hat{y} = 0 \text{ si } p < 0,5$$

Coste log-loss:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log p^{(i)} + (1 - y^{(i)}) \log(1 - p^{(i)}) \right]$$

Trampa típica: La regresión logística no predice valores continuos como tarea principal; predice probabilidades de clase.

Softmax

Extiende la regresión logística a clasificación multiclase excluyente.

$$s_k(x) = \theta_k^T x, \quad p_k = \frac{e^{s_k(x)}}{\sum_j e^{s_j(x)}}$$

La predicción es:

$$\hat{y} = \arg \max_k p_k$$

k vecinos más cercanos

kNN es aprendizaje basado en instancias. Memoriza ejemplos y predice por similitud.

Para clasificación, predice por mayoría entre los k vecinos más cercanos. Para regresión, usa la media de sus valores.

Distancias habituales:

- Euclídea, norma L_2 .
- Manhattan, norma L_1 .
- Hamming para atributos nominales.
- Mahalanobis cuando importa la covarianza entre variables.

Trampa típica: kNN casi no tiene entrenamiento, pero puede ser caro en predicción. Además, es muy sensible a la escala; normalmente hay que normalizar.

Naive Bayes

Usa el teorema de Bayes y asume independencia condicional de atributos dada la clase.

$$c_{MAP} = \arg \max_c P(c) \prod_i P(a_i | c)$$

Con variables discretas se estiman probabilidades por frecuencia. El suavizado de Laplace evita probabilidades cero:

$$P(x_i | c_j) = \frac{n(x_i, c_j) + 1}{n(c_j) + |\Omega_{X_i}|}$$

Trampa típica: La independencia asumida rara vez es totalmente cierta, pero el método puede funcionar bien.

SVM

Las máquinas de vectores soporte buscan un hiperplano con margen máximo.

Función de decisión lineal:

$$s = w^T x + b$$

Los **vectores de soporte** son los puntos que determinan el margen.

Margen duro: exige separación perfecta, sensible a outliers y solo aplicable si los datos son linealmente separables.

Margen blando: permite violaciones mediante holguras. El hiperparámetro C controla el equilibrio:

- C alto: menos violaciones, margen más estrecho, más riesgo de sobreajuste.
- C bajo: más regularización, margen más ancho, más tolerancia a errores.

Kernel trick

Permite trabajar en espacios transformados sin calcular explícitamente $\phi(x)$.

$$K(a, b) = \phi(a)^T \phi(b)$$

Kernels frecuentes:

- Lineal: $K(a, b) = a^T b$.
- Polinómico: $K(a, b) = (a^T b)^d$.
- RBF: $K(a, b) = e^{-\gamma \|a-b\|^2}$.

Árboles de decisión

Modelos interpretables que dividen recursivamente los datos según atributos.

Entropía:

$$H(S) = - \sum_i p_i \log_2(p_i)$$

Ganancia de información:

$$Gain(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$$

Gini:

$$Gini(T) = 1 - \sum_j p_j^2$$

- ID3 usa ganancia de información.
- C4.5 usa gain ratio.
- CART usa divisiones binarias y Gini para clasificación o MSE para regresión.

Trampa típica: Los árboles son interpretables, pero sobreajustan con facilidad si no se podan o regularizan.

Ensembles

Combinan varios modelos para mejorar rendimiento.

Hard voting: gana la clase más votada. **Soft voting:** se promedian probabilidades.

Bagging: entrena modelos en muestras con reemplazo. Reduce varianza y se paraleliza bien. Random Forest es bagging de árboles más aleatoriedad en características.

Out-of-bag: en bagging, aproximadamente el 37 % de las instancias queda fuera de cada muestra bootstrap y puede usarse para estimar rendimiento.

Boosting: modelos secuenciales donde cada nuevo predictor corrige errores del anterior. Ejemplos: AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost.

Stacking: un meta-modelo aprende a combinar las predicciones de modelos base.

Trampa típica: Bagging reduce varianza y puede paralelizarse; boosting es secuencial y corrige errores.

Clasificación multiclase

One-vs-Rest: para c clases entrena c clasificadores. Cada uno distingue una clase contra el resto.

One-vs-One: para c clases entrena $c(c-1)/2$ clasificadores. Para MNIST con 10 clases: $10 \cdot 9 / 2 = 45$ clasificadores.

Matriz de confusión y métricas

	Real positivo	Real negativo
Predicho positivo	TP	FP
Predicho negativo	FN	TN

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

$$Specificity = \frac{TN}{TN + FP}, \quad FPR = \frac{FP}{FP + TN} = 1 - Specificity$$

Curvas ROC y PR

La curva ROC representa TPR frente a FPR . El AUC mide área bajo la curva: 1 perfecto, 0.5 azar.

En clases muy desbalanceadas suele ser más informativa la curva precisión-recall.

Trampa típica: Accuracy puede engañar con clases desbalanceadas. Si el 99 % son negativos, un clasificador que siempre predice negativo tiene 99 % accuracy y cero utilidad para detectar positivos.

Regularización lineal y sesgo-varianza

El error de generalización puede verse como suma de sesgo, varianza y ruido irreducible.

- Alto sesgo: modelo demasiado simple, infraajuste.
- Alta varianza: modelo demasiado sensible al entrenamiento, sobreajuste.

Ridge:

$$J(\theta) = MSE + \alpha \|w\|_2^2$$

Lasso:

$$J(\theta) = MSE + \alpha \|w\|_1$$

Elastic Net mezcla L1 y L2. Lasso puede poner pesos exactamente a cero, actuando como selección de características.

Regresión polinómica

La **regresión polinómica** permite ajustar relaciones no lineales usando un modelo lineal sobre características transformadas.

Ejemplo con una variable:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$$

El modelo sigue siendo lineal respecto a los parámetros θ , aunque la curva respecto a x sea no lineal. En la práctica se crean nuevas columnas como x^2 , x^3 o productos entre variables, y después se aplica regresión lineal.

Trampa típica: Polinómico no significa que el algoritmo deje de ser lineal en los parámetros. Es lineal en θ , no necesariamente en las variables originales.

Descenso de gradiente en modelos lineales

Cuando no se usa ecuación normal, se pueden ajustar parámetros minimizando una función de coste iterativamente.

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$$

Tipos:

- **Batch Gradient Descent:** calcula el gradiente con todo el entrenamiento. Es estable, pero caro con muchos datos.
- **Stochastic Gradient Descent:** usa una instancia por actualización. Es rápido y ruidoso; puede escapar de mínimos malos, pero oscila.
- **Mini-batch Gradient Descent:** usa pequeños lotes; equilibrio habitual entre velocidad y estabilidad.

La tasa de aprendizaje η es crítica: si es muy baja, tarda mucho; si es muy alta, puede divergir. El escalado de características es importante porque facilita que el descenso avance de forma equilibrada.

Curvas de aprendizaje

Las **curvas de aprendizaje** representan el error de entrenamiento y validación según aumenta el tamaño del conjunto de entrenamiento o el número de iteraciones.

Interpretación típica:

- **Infraajuste:** error alto en entrenamiento y validación. El modelo es demasiado simple o faltan características útiles.
- **Sobreaajuste:** error bajo en entrenamiento y alto en validación. El modelo memoriza demasiado.
- **Buen ajuste:** errores bajos y cercanos.

Equilibrio sesgo-varianza

El **sesgo** mide error por hipótesis demasiado simples. La **varianza** mide sensibilidad excesiva a los datos de entrenamiento.

Regla de examen:

- Aumentar complejidad suele bajar sesgo y subir varianza.
- Reducir complejidad suele subir sesgo y bajar varianza.
- Regularizar reduce varianza, pero si se exagera puede aumentar demasiado el sesgo.

Modelos lineales regularizados en detalle

Ridge añade penalización L2. No se suele regularizar el término de sesgo θ_0 .

$$J(\theta) = MSE + \alpha \sum_{i=1}^n \theta_i^2$$

Si $\alpha = 0$, Ridge equivale a regresión lineal. Si α es grande, los pesos se acercan a cero, pero normalmente no se hacen exactamente cero.

Lasso añade penalización L1.

$$J(\theta) = MSE + \alpha \sum_{i=1}^n |\theta_i|$$

Lasso puede dejar pesos exactamente a cero, por eso actúa como selección automática de características.

Elastic Net mezcla Ridge y Lasso.

$$J(\theta) = MSE + r\alpha \sum_i |\theta_i| + \frac{1-r}{2}\alpha \sum_i \theta_i^2$$

Elastic Net suele ser preferible a Lasso cuando hay muchas variables correlacionadas o más variables que instancias.

Parada anticipada o Early Stopping

La **parada anticipada** detiene el entrenamiento cuando el error de validación deja de mejorar.

Procedimiento:

1. Entrenar durante varias épocas o iteraciones.
2. Medir pérdida en validación.
3. Guardar el modelo con mejor validación.
4. Parar cuando la validación empeora durante cierto número de iteraciones.

Es una forma de regularización porque evita seguir ajustando el ruido del entrenamiento.

Regresión logística: pérdida e interpretación

La pérdida por instancia se define como:

$$L(p, y) = \begin{cases} -\log(p), & y = 1 \\ -\log(1-p), & y = 0 \end{cases}$$

Si el ejemplo real es positivo y el modelo asigna p pequeño, $-\log(p)$ se dispara. Si el ejemplo real es negativo y el modelo asigna p grande, $-\log(1-p)$ se dispara.

No existe una ecuación normal cerrada como en regresión lineal, pero la función de coste es convexa y puede optimizarse con descenso de gradiente.

Softmax: entrenamiento multiclase

En softmax cada clase k tiene un vector de parámetros $\theta^{(k)}$. Todos los vectores forman una matriz Θ .

Los valores $s_k(x)$ se llaman **logits**. Softmax los transforma en probabilidades que suman 1.

La pérdida habitual es entropía cruzada:

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(p_k^{(i)})$$

Si hay solo dos clases, softmax se reduce conceptualmente a regresión logística binaria.

Compromiso precisión-recall

Muchos clasificadores calculan una puntuación y comparan con un umbral.

- Subir el umbral: menos positivos predichos, suele subir precisión y bajar recall.
- Bajar el umbral: más positivos predichos, suele subir recall y bajar precisión.

La curva precisión-recall muestra cómo cambian ambas métricas al variar el umbral. Es especialmente útil con clases desbalanceadas.

Notación y normas en regresión

En regresión, m es el número de instancias, $x^{(i)}$ es la instancia i , $y^{(i)}$ su valor real y $\hat{y}^{(i)} = h(x^{(i)})$ la predicción.

La matriz X contiene una fila por instancia y una columna por característica. La hipótesis h es el modelo aprendido.

Relación con normas:

- RMSE se asocia a norma L_2 o euclídea.
- MAE se asocia a norma L_1 o Manhattan.
- La norma L_∞ mira el mayor error absoluto.

Scikit-Learn: API básica

Scikit-Learn usa una API muy regular:

- **Estimador**: aprende con `fit()`.
- **Transformador**: transforma con `transform()` y suele tener `fit_transform()`.
- **Predictor**: predice con `predict()` y evalúa con `score()`.
- Los hiperparámetros son atributos públicos normales; los parámetros aprendidos suelen acabar en guion bajo, como `coef_`.
- **Pipeline** encadena transformadores y modelo final.

Aprendizaje basado en instancias frente a modelos

Basado en instancias: memoriza ejemplos y generaliza por similitud. kNN es el caso típico.

Basado en modelos: aprende parámetros de una función y luego predice con esa función. Regresión lineal, SVM, árboles y redes son ejemplos.

kNN ampliado

En 1-NN se busca la instancia de entrenamiento más cercana:

$$x^* = \arg \min_{x_i} d(x, x_i)$$

y se devuelve su clase o valor.

Distancias adicionales:

- Chebyshev o L_∞ : máximo de las diferencias absolutas.
- Euclídea escalada: pondera características para compensar escalas o importancia.
- Mahalanobis: incorpora la matriz de covarianzas.

La ponderación de características cambia la geometría de las regiones de decisión. Una variable con escala grande puede dominar si no se normaliza.

Naive Bayes con variables numéricas

Para atributos continuos se usa a menudo Naive Bayes gaussiano:

$$P(x | c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{-\frac{(x-\mu_c)^2}{2\sigma_c^2}}$$

Se estima una media y desviación típica por clase y atributo. En variables discretas se usan frecuencias y suavizado de Laplace.

SVM ampliado

En margen duro, para etiquetas $t^{(i)} \in \{-1, 1\}$:

$$t^{(i)}(w^T x^{(i)} + b) \geq 1$$

El entrenamiento busca maximizar el margen, equivalente a minimizar:

$$\frac{1}{2} \|w\|^2$$

sujeto a las restricciones.

En margen blando se introducen holguras $\xi^{(i)}$ para permitir errores. La pérdida hinge penaliza violaciones del margen:

$$\max(0, 1 - t^{(i)}(w^T x^{(i)} + b))$$

SVM no lineal: puede usar características polinómicas explícitas o kernels. El kernel RBF equivale a medir similitud con puntos de referencia en un espacio de alta dimensión.

Regresión SVM: en lugar de separar clases, intenta meter el máximo de puntos dentro de una calle de anchura ϵ . Los errores dentro de la calle no penalizan.

Regla práctica: probar primero un SVM lineal si hay muchas instancias o características; si el dataset no es enorme, probar RBF.

Árboles de decisión ampliados

Un árbol se puede convertir en reglas **if-then**. Ejemplo: si **humedad=alta** y **tiempo=soleado**, entonces **jugar=no**.

Construcción general:

1. Elegir la mejor división según impureza o ganancia.
2. Dividir los datos.
3. Repetir recursivamente.
4. Parar si el nodo es puro, no quedan atributos, no hay mejora o se alcanza una restricción.

ID3 usa ganancia de información y puede favorecer atributos con muchos valores. C4.5 corrige con **gain ratio**, maneja atributos continuos, valores perdidos y poda. CART usa divisiones binarias por característica y umbral.

En una hoja, la probabilidad de clase puede estimarse como proporción de ejemplos de esa clase en la hoja. En regresión, el árbol predice normalmente la media de los valores de la hoja.

Hiperparámetros de regularización:

- **max_depth**
- **min_samples_split**
- **min_samples_leaf**
- **max_leaf_nodes**
- **max_features**

Gini suele ser algo más rápido; entropía puede producir árboles algo más equilibrados, aunque normalmente son parecidos.

Limitaciones: alta varianza, sensibilidad a pequeños cambios, fronteras rectangulares paralelas a ejes y dificultad con correlaciones complejas.

Ensembles ampliados

Un ensemble puede convertir aprendices débiles en un predictor fuerte si los modelos son diversos y sus errores no están perfectamente correlacionados.

Pasting: muestreo sin reemplazo. **Bagging**: muestreo con reemplazo. En clasificación se agrega por moda; en regresión por media.

Random Patches: muestrea instancias y características. **Random Subspaces**: muestrea solo características. Son útiles en alta dimensionalidad.

Extra-Trees: árboles extremadamente aleatorios; usan umbrales aleatorios. Suelen entrenar más rápido, con más sesgo y menos varianza.

Random Forest también permite estimar importancia de variables según la reducción media de impureza que produce cada característica.

AdaBoost: aumenta el peso de las instancias mal clasificadas y combina predictores con votos ponderados.

Gradient Boosting: cada nuevo predictor aprende los residuos o errores del conjunto anterior.

Histogram-Based Gradient Boosting: discretiza variables en bins para acelerar; puede manejar variables categóricas y valores perdidos en implementaciones modernas.

Stacking: el mezclador se entrena con predicciones fuera de muestra de los modelos base, para evitar entrenarlo con predicciones demasiado optimistas.

Clasificación multiclase y no balanceada

En OvR cada clasificador ve una clase positiva frente al resto, por lo que puede haber desbalance fuerte. En OvO cada clasificador solo ve dos clases, pero se necesitan $c(c-1)/2$ modelos y pueden aparecer empates.

En clasificación no balanceada, los modelos pueden aprender demasiado la clase mayoritaria e ignorar la minoritaria. Por eso hay que usar métricas como recall, precision, F1, PR-AUC o costes específicos.

Data augmentation

El aumento de datos genera ejemplos artificiales aplicando transformaciones que conservan la etiqueta, como desplazamientos, rotaciones pequeñas o ruido. Mejora robustez y reduce sobreajuste.

Preguntas tipo test del tema 2

1. Precisión responde: de lo predicho positivo, cuánto era realmente positivo.
2. Recall responde: de los positivos reales, cuántos detecta el modelo.
3. F1 es media armónica, no media aritmética.
4. C bajo en SVM implica más regularización.
5. Ridge usa L2; Lasso usa L1 y puede eliminar variables.
6. Softmax se usa para clases excluyentes; sigmoide es típica para salida binaria o multietiqueta.
7. Random Forest no es un único árbol, sino un conjunto de árboles.

Tema 3. Calidad de Datos y Preprocesamiento

Importancia

La calidad de datos determina la calidad del modelo. El preprocesamiento convierte datos brutos en datos útiles, reduce errores y puede consumir gran parte del tiempo de un proyecto real.

Problemas frecuentes:

- Valores perdidos.
- Ruido y etiquetas incorrectas.
- Outliers.
- Duplicados e inconsistencias.
- Escalas distintas.
- Atributos irrelevantes o redundantes.

Preparación y reducción de datos

Preparación: integración, limpieza, normalización, transformación e imputación.

Reducción: selección de características, selección de instancias, discretización y reducción de dimensionalidad.

Valores perdidos

Opciones:

- Eliminar la instancia, especialmente si falta la clase.
- Rellenar con constante: “desconocido”.
- Imputar con media, mediana o moda.
- Imputar por clase.
- Estimar con modelos: kNN, árboles, Bayes, SVM o clustering.

Trampa típica: Imputar con la media puede mantener la media global, pero reduce varianza y puede distorsionar relaciones entre variables.

Ruido y outliers

Ruido: errores, mediciones defectuosas, etiquetas incorrectas o ejemplos contradictorios.

Outlier: dato muy diferente de la mayoría. Puede ser error, fraude, caso raro válido o novedad.

Trampa típica: Outlier no significa automáticamente ruido. Un fraude es outlier, pero puede ser justo lo que interesa detectar.

Filtros de ruido:

- Ensemble Filter: varios clasificadores votan si una instancia parece mal etiquetada.
- CVCF: usa comités por validación cruzada.
- IPF: aplica el filtrado iterativamente.

Normalización

Necesaria en métodos sensibles a escala: kNN, SVM, redes neuronales, PCA y descenso de gradiente.

Min-max:

$$v' = \frac{v - \min_A}{\max_A - \min_A}(\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

Z-score:

$$v' = \frac{v - \mu_A}{\sigma_A}$$

Escala decimal:

$$v' = \frac{v}{10^j}$$

donde j es el menor entero que hace que $\max(|v'|) < 1$.

- Min-max conserva relaciones, pero es sensible a outliers.
- Z-score es útil si no se conocen límites o hay valores extremos.

Discretización

Convierte variables continuas en intervalos nominales. Ejemplo: edad → joven/adulto/mayor.

Métodos no supervisados:

- Igual amplitud: intervalos del mismo ancho.
- Igual frecuencia: intervalos con número parecido de ejemplos.
- Clustering.

Métodos supervisados:

- Fayyad-Irani: basado en entropía y Minimum Description Length.
- ChiMerge: basado en chi-cuadrado.

Trampa típica: Igual amplitud puede dar intervalos muy desequilibrados si hay valores extremos.

Selección de características

Busca un subconjunto de variables útil para predecir, reduciendo tiempo, coste, sobreajuste e interpretabilidad.

Con N variables hay 2^N subconjuntos posibles. Por eso se usan búsquedas heurísticas.

Filtros: evalúan variables sin entrenar el clasificador final. Son rápidos y generales.

Wrappers: prueban subconjuntos usando el clasificador final. Son más adaptados, pero costosos y pueden sobreajustar.

Embedded: la selección ocurre durante el entrenamiento. Ejemplos: Lasso, árboles.

Búsquedas:

- Forward selection: empieza vacío y añade variables.
- Backward selection: empieza con todas y elimina variables.
- Estocásticas: algoritmos genéticos, enfriamiento simulado, reinicios.

Trampa típica: Selección de características reduce columnas; selección de instancias reduce filas.

Selección de instancias

Elige ejemplos representativos para reducir almacenamiento y tiempo, y mejorar generalización.

Ejemplos:

- CNN, Condensed Nearest Neighbor: añade instancias mal clasificadas y retiene puntos de frontera.
- ENN, Edited Nearest Neighbor: elimina instancias mal clasificadas por sus vecinos, suavizando fronteras.
- DROP3: método híbrido usado con kNN.

Reducción de dimensionalidad

La maldición de la dimensionalidad provoca datos dispersos, distancias menos fiables, más sobreajuste y más coste.

PCA: técnica no supervisada que proyecta los datos sobre componentes principales de máxima varianza. No usa etiquetas.

LDA: técnica supervisada que busca ejes que separen clases.

t-SNE: útil para visualización, preserva vecindades locales.

Trampa típica: PCA preserva varianza, no necesariamente separabilidad entre clases.

Integración de datos

La **integración** combina datos procedentes de varias fuentes, bases de datos o sistemas. Es frecuente en data warehouses y proyectos reales.

Problemas típicos:

- Atributos equivalentes con unidades distintas, como salario mensual y salario anual.
- Duplicados de instancias o variables.

- Inconsistencias entre fuentes.
- Atributos redundantes que pueden derivarse de otros.
- Correlaciones fuertes que indican información repetida.

Limpieza y transformación

La **limpieza** busca resolver inconsistencias, suavizar ruido e identificar o eliminar outliers cuando sean errores.

La **transformación** adapta los datos al algoritmo:

- Agregación: ventas diarias $\rightarrow$ ventas mensuales.
- Generalización: calle $\rightarrow$ ciudad; edad $\rightarrow$ joven/adulto/mayor.
- Normalización y estandarización.
- Transformaciones lineales, cuadráticas, logarítmicas o polinómicas.

Ventajas del preprocesamiento: mejora precisión, reduce tiempo, permite aplicar algoritmos a más datos, mejora interpretabilidad y ayuda a comprender el dominio.

Imputación avanzada

Además de eliminar o imputar con media, se puede:

- Rellenar manualmente, aunque suele ser caro.
- Imputar preservando media y desviación típica.
- Usar relaciones entre variables, por ejemplo una regresión entre columnas.
- Usar KNNI, WKNNI, KMI, SVMi, SVDI o LLSI.

Trampa típica: Una imputación simple puede introducir sesgo. Si faltan valores de forma no aleatoria, imputar sin analizar puede distorsionar el dataset.

Ruido en clasificación

En clasificación se distinguen ejemplos:

- **Safe:** lejos de la frontera de decisión.
- **Borderline:** cerca de la frontera.
- **Noisy:** mal etiquetados, contradictorios o muy alejados de su clase.

También pueden aparecer **small disjuncts**, pequeñas regiones de una clase con pocos ejemplos, difíciles de distinguir de ruido.

Filtros de ruido en detalle

Ensemble Filter: usa varios clasificadores, como C4.5, 1-NN y LDA, normalmente con validación cruzada. Puede eliminar ejemplos por:

- Consensus voting: todos los clasificadores lo consideran mal.
- Majority voting: la mayoría lo considera mal.

CVCF: usa comités validados cruzadamente con el mismo algoritmo y etiqueta todos los ejemplos.

IPF: aplica filtrado iterativamente hasta que el número de ruidosos detectados cae bajo un umbral durante varias iteraciones.

Detección de outliers

Métodos simples:

- Calcular distancia media de cada punto al resto y marcar los más alejados.
- Usar clustering parcial: puntos fuera de clusters densos pueden ser atípicos.

Los outliers perjudican especialmente métodos sensibles al ajuste de pesos o distancias.

Discretización ampliada

La discretización produce valores más concisos y comprensibles. También es reducción de datos porque convierte dominios continuos en dominios finitos.

Taxonomía:

- Supervisada o no supervisada.
- Dinámica o estática.
- Local o global.
- Top-down o bottom-up.
- Directa o incremental.

MDLP de Fayyad-Irani: parte de puntos de corte entre ejemplos de distinta clase, usa entropía y acepta cortes si reducen la longitud de descripción de los datos.

Criterios de evaluación de discretizadores:

- Número total de intervalos.
- Inconsistencias introducidas.
- Tasa de acierto predictivo.

Selección de características ampliada

La selección busca un subconjunto que maximice rendimiento predictivo y reduzca coste.

Componentes:

- Estrategia de búsqueda.
- Función objetivo.

Medidas filtro:

- Separabilidad entre clases: distancias euclídeas o Mahalanobis.
- Correlación con la clase y baja redundancia entre variables.
- Información mutua: reducción de incertidumbre sobre la clase.
- Consistencia: mínimo subconjunto que conserva capacidad de distinguir clases.

Los filtros pueden devolver un ranking o un subconjunto. Los wrappers son costosos y dependientes del clasificador. Los filtros pueden incluir demasiadas variables si la función objetivo es monótona.

Algoritmos frecuentes: l -más r -menos, búsqueda bidireccional, selección flotante, branch and bound, beam search, Focus, MIFS, LVF, LVW, Relief y mRMR.

Extracción y construcción de características

La **selección** escoge variables existentes. La **extracción** o construcción crea nuevas variables, por ejemplo productos, combinaciones o proyecciones. En árboles, una combinación de atributos puede

simplificar fronteras que serían difíciles con variables originales.

Selección de instancias ampliada

Puede entenderse como muestreo, selección de prototipos o aprendizaje activo.

Criterios:

- Reducir almacenamiento.
- Mantener o mejorar generalización.
- Tolerar ruido.
- Reducir cómputo.

Propiedades:

- Dirección: incremental, decremental, por lotes, mezclada o fijada.
- Tipo: condensación, edición o híbrido.
- Evaluación: filtro o wrapper.

CNN es incremental, depende del orden y retiene puntos de frontera. ENN es por lotes, suele usar $k = 3, 5, 9$, suaviza fronteras pero deja puntos redundantes.

Otros métodos: AllKNN, RMHC, SSMA, Multiedit, IB2, IB3, ICF, CHC, DROP1, DROP2 y DROP3.

En grandes bases de datos estos métodos pueden ser caros, a menudo $O(n^2)$ o $O(n^3)$. Una estrategia es estratificar, aplicar selección por estratos y combinar.

Maldición de la dimensionalidad

En alta dimensión los datos se vuelven dispersos y las distancias pierden significado. Muchos puntos quedan cerca de la frontera del espacio y las predicciones dependen más de extrapolaciones. Mantener densidad requiere un número de instancias que crece exponencialmente.

Proyección, manifolds y PCA

La reducción de dimensionalidad puede hacerse por **proyección** a un subespacio o por **manifold learning**, que intenta descubrir variedades no lineales.

PCA elige componentes principales ortogonales de máxima varianza. Puede obtenerse con SVD. Para reducir a d dimensiones se proyecta sobre los d primeros componentes. La reconstrucción desde menos dimensiones introduce **error de reconstrucción**.

Reducir dimensión suele acelerar, pero no garantiza mejorar precisión.

PCA incremental, proyección aleatoria y LLE

PCA incremental permite procesar datos por lotes cuando no caben en memoria o llegan online.

Proyección aleatoria es rápida y puede preservar distancias aproximadamente.

LLE: Locally Linear Embedding. Técnica no lineal que conserva relaciones locales entre vecinos, útil para manifolds retorcidos.

Otras técnicas:

- MDS: preserva distancias.
- Isomap: preserva distancias geodésicas.

- t-SNE: visualización de vecindades.
- LDA: proyección supervisada que maximiza separación entre clases.

Preguntas tipo test del tema 3

1. Min-max es sensible a outliers.
2. Z-score usa media y desviación típica.
3. Outlier y ruido no son sinónimos.
4. Filtros son más rápidos que wrappers, pero menos adaptados al modelo final.
5. PCA es no supervisado; LDA es supervisado.
6. Discretización transforma valores continuos en intervalos.
7. Selección de instancias reduce ejemplos; selección de características reduce atributos.

Tema 4. Aprendizaje No Supervisado

Idea general

Trabaja con datos sin etiquetas y busca estructura oculta: clusters, anomalías, densidades, dimensiones latentes o reglas de asociación.

Clustering

Agrupar instancias similares. Aplicaciones: segmentación de clientes, compresión, exploración de datos, segmentación de imágenes, recomendadores, detección de anomalías.

K-Means

Algoritmo:

1. Elegir k centroides iniciales.
2. Asignar cada punto al centroide más cercano.
3. Actualizar cada centroide como la media de sus puntos.
4. Repetir hasta convergencia.

Inercia: suma de distancias cuadradas de cada punto a su centroide más cercano.

Trampa típica: La inercia siempre baja al aumentar k , así que no basta para elegir el número de clusters.

Elección de k

Método del codo: elegir el punto donde la mejora de inercia empieza a reducirse.

Coeficiente de silueta:

$$s = \frac{b - a}{\max(a, b)}$$

donde a es la distancia media al propio cluster y b al cluster vecino más cercano.

Interpretación:

- Cerca de 1: bien asignado.
- Cerca de 0: en frontera.

- Cerca de -1: probablemente mal asignado.

Limitaciones de K-Means

Funciona mejor con clusters aproximadamente esféricos, tamaños similares y densidades parecidas. Falla con formas alargadas, densidades distintas, outliers fuertes o k incorrecto.

DBSCAN

Algoritmo basado en densidad.

Hiperparámetros:

- ε o eps: radio de vecindad.
- min_samples: número mínimo de puntos para considerar zona densa.

Conceptos:

- Punto núcleo: tiene al menos min_samples en su vecindad.
- Punto frontera: no es núcleo, pero cae en vecindad de uno.
- Ruido: no pertenece a ningún cluster.

Ventajas: no exige fijar número de clusters, detecta formas arbitrarias y ruido. Limitación: falla con densidades muy diferentes y no escala bien a datasets enormes.

Clustering jerárquico

Construye una jerarquía de clusters. En aglomerativo empieza con cada punto como cluster y fusiona los clusters más cercanos. Produce un dendrograma.

Trampa típica: Jerárquico no busca centroides como K-Means; construye una estructura de fusiones.

Otros métodos

- BIRCH: diseñado para datasets grandes.
- Mean-Shift: busca modos de densidad, no requiere k , pero puede ser costoso.
- Affinity Propagation: elige exemplars, no requiere k , pero escala mal.
- Clustering espectral: usa matriz de similitud y embeddings; útil para estructuras complejas.

Detección de anomalías

Busca casos raros o inusuales. Ejemplos: fraude, intrusiones, fallos, enfermedades raras, productos defectuosos.

Tipos:

- Puntual: una instancia aislada es rara.
- Contextual: es rara solo en cierto contexto. Ejemplo: 30 grados en invierno.
- Colectiva: una secuencia o conjunto es raro aunque cada punto individual no lo parezca.

Métodos:

- kNN: anomalías lejos de sus vecinos.

- LOF: compara densidad local con densidad de vecinos.
- One-Class SVM: aprende frontera de normalidad.
- Isolation Forest: las anomalías se aíslan con pocos cortes, por eso tienen caminos cortos.

Reglas de asociación

Descubren relaciones frecuentes entre elementos. Ejemplo: quien compra salsa barbacoa y patatas puede comprar filetes.

Métricas:

- Soporte: frecuencia de aparición.
- Confianza: probabilidad del consecuente dado el antecedente.
- Lift: cuánto aumenta la probabilidad respecto a la frecuencia base.

Trampa típica: Una regla de asociación no implica causalidad.

Estimación de densidad

La **estimación de densidad** intenta modelar dónde se concentran los datos en el espacio. Las regiones de baja densidad suelen ser candidatas a anomalías.

Usos:

- Detección de anomalías.
- Visualización de estructura.
- Entender distribución de datos sin etiquetas.

Clustering como representación

Además de asignar grupos, el clustering puede transformar una instancia en un vector de afinidades a clusters. Si hay k clusters, una instancia puede representarse por k distancias o similitudes a centroides.

Aplicaciones ampliadas:

- Aprendizaje semisupervisado: propagar etiquetas dentro de clusters.
- Ingeniería de características: usar afinidades como nuevas variables.
- Motores de búsqueda: agrupar documentos similares.
- Segmentación de imágenes: reemplazar píxeles por el color medio del cluster.

K-Means ampliado

K-Means puede converger a óptimos locales, por eso importa la inicialización y se ejecuta varias veces.

K-Means++:

1. Elige el primer centroide al azar.
2. Elige los siguientes con probabilidad mayor cuanto más lejos estén de los centroides ya elegidos.
3. Reduce la probabilidad de malas inicializaciones.

Mini-Batch K-Means: usa minilotes en vez de todo el dataset. Es más rápido y sirve para datos enormes, con una inercia a veces algo peor.

K-Means acelerado de Elkan: usa desigualdad triangular y cotas de distancia para evitar cálculos innecesarios. Puede acelerar, pero no siempre.

Complejidad: suele comportarse casi linealmente en número de instancias m , clusters k y dimensiones n , aunque el peor caso puede ser mucho peor.

Métricas de calidad de clustering

Además de silueta:

- **Davies-Bouldin:** compara dispersión interna con separación entre clusters. Menor es mejor.
- **Dunn index:** distancia mínima entre clusters dividida por diámetro máximo interno. Mayor es mejor.
- **Pseudo-F:** compara variabilidad entre clusters frente a variabilidad dentro de clusters. Mayor es mejor.

DBSCAN ampliado

DBSCAN expande clusters a partir de puntos núcleo conectados por cadenas de vecindades densas.

Pasos:

1. Buscar puntos con al menos `min_samples` dentro de radio ϵ .
2. Marcar esos puntos como núcleos.
3. Expandir clusters conectando núcleos alcanzables.
4. Puntos no alcanzables quedan como ruido.

Funciona bien cuando los clusters están separados por regiones de baja densidad. Su complejidad puede aproximarse a $O(m^2n)$ si no hay estructuras de índice eficientes.

HDBSCAN extiende DBSCAN jerárquicamente y suele manejar mejor densidades distintas.

Clustering aglomerativo ampliado

El clustering aglomerativo genera un árbol binario de fusiones. Las hojas son instancias individuales y la raíz representa todos los datos. Puede usar distintas distancias entre clusters y dendrogramas.

Una matriz de conectividad puede limitar fusiones a vecinos y mejorar escalabilidad. Sin ella, puede escalar mal en datasets grandes.

BIRCH, Mean-Shift, Affinity Propagation y espectral

BIRCH resume datos en un árbol compacto, útil con memoria limitada y datasets grandes. Funciona mejor con pocas características, típicamente menos de unas 20.

Mean-Shift desplaza ventanas hacia máximos locales de densidad. No requiere k , pero depende de `bandwidth` y puede ser $O(m^2n)$.

Affinity Propagation intercambia mensajes entre instancias y elige representantes o *exemplars*. No requiere k , pero su coste $O(m^2)$ limita datasets grandes.

Clustering espectral usa una matriz de similitud, obtiene una representación de baja dimensión y luego agrupa. Es útil para grafos y formas complejas, pero escala mal y puede fallar con tamaños de cluster muy distintos.

Retos en detección de anomalías

Detectar anomalías es difícil porque:

- La normalidad puede ser difusa.
- Hay pocas etiquetas de anomalías.
- El comportamiento normal cambia con el tiempo.
- Puede haber ruido y adversarios.
- Depende mucho del dominio.

Taxonomía de métodos:

- Basados en clasificación.
- Basados en vecinos.
- Basados en clustering.
- Estadísticos o de densidad.
- Basados en teoría de la información.
- Descomposición espectral y visualización.
- Online o distribuidos.

Métodos de anomalías ampliados

kNN para anomalías: calcula la distancia al k -ésimo vecino más cercano. Los puntos con mayor distancia se marcan como outliers. Falla con densidades variables.

LOF: compara densidad local de un punto con la densidad de sus vecinos. Detecta mejor anomalías locales en regiones con distintas densidades.

One-Class SVM: aprende una frontera que separa la clase normal del origen con máximo margen. Si $f(x) < 0$, se considera anomalía. También se relaciona con encontrar una hiperesfera mínima que describa los datos normales.

Isolation Forest: construye muchos árboles aleatorios. En cada nodo elige una característica y un corte aleatorio. Las anomalías suelen aislarse con menos cortes, por eso tienen profundidad media menor. El score final promedia la profundidad entre árboles y se aplica un umbral.

Preguntas tipo test del tema 4

1. K-Means necesita fijar k antes de entrenar.
2. K-Means++ mejora la inicialización de centroides.
3. DBSCAN no necesita k , pero depende de ϵ y min_samples .
4. Silueta negativa sugiere mala asignación.
5. Isolation Forest detecta outliers porque se aíslan antes.
6. PCA no agrupa: transforma variables.
7. Asociación no equivale a causalidad.

Tema 5. Aprendizaje Profundo: Fundamentos

Perceptrón

Una neurona lineal calcula:

$$z = w^T x + b$$

y aplica una función escalón:

$$salida = step(z)$$

El perceptrón simple define una frontera lineal. Si los datos son linealmente separables, converge. No puede resolver XOR con una sola capa.

MLP

Un perceptrón multicapa tiene capas de entrada, ocultas y salida. Las capas ocultas permiten no linealidad y representaciones jerárquicas.

Funciones de activación

Sin activaciones no lineales, varias capas lineales equivalen a una sola transformación lineal.

Función	Uso típico
Sigmoide	Salida binaria, probabilidad entre 0 y 1
Tanh	Similar a sigmoide pero centrada en 0
ReLU	Buen valor por defecto en capas ocultas
Leaky ReLU	Reduce neuronas ReLU muertas
ELU/SELU	Ayudan a estabilizar activaciones
GELU/Swish	Frecuentes en redes profundas modernas
Softmax	Salida multiclase excluyente
Lineal	Regresión

Backpropagation

Backpropagation combina forward pass, cálculo de pérdida, regla de la cadena, cálculo de gradientes y actualización de parámetros.

Actualización básica:

$$\theta \leftarrow \theta - \eta \nabla J(\theta)$$

donde η es la tasa de aprendizaje.

Trampa típica: La función escalón no sirve para backprop porque no da gradientes útiles.

Batch, mini-batch y época

- Batch completo: calcula gradiente con todos los datos.
- SGD estricto: usa una instancia por actualización.
- Mini-batch: usa subconjuntos; es el estándar práctico.
- Época: una pasada completa por el conjunto de entrenamiento.

Inicialización y gradientes

Problemas:

- Gradientes desaparecidos: capas inferiores apenas aprenden.
- Gradientes explosivos: actualizaciones enormes y divergencia.

Soluciones:

- Glorot/Xavier para activaciones simétricas.
- He/Kaiming para ReLU.
- LeCun para SELU.
- Batch Normalization.
- Gradient clipping.

Trampa típica: Inicializar todos los pesos a cero rompe la simetría: neuronas iguales aprenden lo mismo.

Batch Normalization

Normaliza entradas de capas usando estadísticas del mini-batch y aprende escala γ y desplazamiento β .

Ventajas: acelera entrenamiento, estabiliza gradientes y puede regularizar ligeramente.

En inferencia usa estadísticas acumuladas, no las del batch actual.

Regularización y dropout

Dropout apaga neuronas aleatoriamente durante entrenamiento. Reduce coadaptación y se parece a entrenar muchas subredes que comparten pesos.

- En entrenamiento se aplica dropout.
- En test normalmente no se aplica.
- No se suele aplicar a la capa de salida.

Otras técnicas contra sobreajuste: early stopping, L1, L2, max-norm, data augmentation, transfer learning, reducir arquitectura o conseguir más datos.

Optimizadores

- Momentum: acumula velocidad y atraviesa mesetas.
- Nesterov: mira un poco por delante.
- AdaGrad: adapta tasa por parámetro, pero puede frenarse demasiado.
- RMSProp: usa media exponencial de gradientes cuadrados.
- Adam: combina momentum y RMSProp.
- AdamW: Adam con weight decay desacoplado, suele generalizar mejor que Adam con L2 simple.

Transfer learning

Reutiliza capas de un modelo entrenado en una tarea relacionada. Procedimiento típico: sustituir capa final, congelar capas base, entrenar nuevas capas y luego hacer fine-tuning con tasa menor.

Salidas según tarea

Tarea	Salida	Pérdida típica
Regresión	Lineal	MSE o MAE
Binaria	Sigmoide	Entropía cruzada binaria
Multiclase excluyente	Softmax	Entropía cruzada categórica
Multietiqueta	Sigmoides independientes	Entropía cruzada binaria por etiqueta

Contexto histórico y motivación biológica

Las redes neuronales artificiales se inspiran de forma simplificada en neuronas biológicas: señales, conexiones, sinapsis y activación. No son una copia fiel del cerebro, sino una abstracción matemática.

Hitos:

- McCulloch y Pitts: neuronas binarias y lógica proposicional.
- Hebb: aprendizaje reforzando conexiones entre neuronas que se activan juntas.
- Rosenblatt: perceptrón.
- Minsky y Papert: limitaciones como XOR, que contribuyeron al invierno de las redes neuronales.
- Resurgimiento: más datos, GPUs, mejores activaciones, mejores inicializaciones, backprop eficiente y financiación.

Perceptrón ampliado

Un perceptrón puede tener varias TLU en una capa densa. Con varias salidas, realiza varias clasificaciones binarias en paralelo.

Forma matricial simplificada:

$$Z = XW + b$$

La regla de aprendizaje ajusta pesos cuando hay error:

$$w \leftarrow w + \eta(y - \hat{y})x$$

Es similar a un clasificador lineal: se parece a regresión logística en que calcula una combinación lineal, pero usa función escalón en vez de sigmoide; y comparte con SVM lineal la limitación de fronteras lineales.

Backpropagation y diferenciación automática

Backpropagation usa **diferenciación automática en modo inverso**. Guarda activaciones intermedias durante la pasada hacia delante, calcula la pérdida y propaga contribuciones de error hacia atrás.

No es exclusiva de redes neuronales: sirve para grafos computacionales diferenciables. En redes profundas permite calcular todos los gradientes de pesos y sesgos con dos recorridos principales: hacia delante y hacia atrás.

La función escalón se sustituyó por activaciones diferenciables porque sus segmentos planos tienen derivada nula y no permiten aprender por gradiente.

Funciones de activación ampliadas

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = 2\sigma(2z) - 1$$

$$\text{ReLU}(z) = \max(0, z), \quad \text{LeakyReLU}(z) = \max(\alpha z, z)$$

$$\text{Swish}(z) = z\sigma(\beta z)$$

ReLU puede sufrir **dying ReLU**: una neurona queda siempre en la zona negativa y deja de aprender. Suele ocurrir con tasas de aprendizaje altas. Leaky ReLU, PReLU y ELU reducen este riesgo.

PReLU aprende α , pero puede sobreajustar en datasets pequeños. RReLU usa pendiente aleatoria durante entrenamiento. ELU toma valores negativos, centra activaciones y puede tener gradiente no nulo para $z < 0$. SELU puede autonormalizar MLP densos profundos si se cumplen condiciones estrictas de inicialización y arquitectura.

Regla práctica: ReLU para tareas sencillas y baja latencia; Swish/GELU para modelos más complejos; SELU en MLP profundos si se respetan sus condiciones.

MLP para regresión y clasificación

En regresión con una salida numérica se usa una neurona lineal. Para regresión multisalida se usan varias neuronas, por ejemplo coordenadas de una caja: x , y , anchura y altura.

En clasificación binaria se usa una neurona sigmoide. En multiclase excluyente se usa una neurona por clase y softmax, que produce probabilidades entre 0 y 1 que suman 1. En multietiqueta se usan sigmoides independientes.

Ecosistema y Keras

Herramientas frecuentes: TensorFlow, Keras, PyTorch y JAX. Keras es una API de alto nivel para construir, compilar, entrenar, evaluar y predecir con redes.

Flujo típico:

1. Preparar datos, por ejemplo Fashion MNIST.
2. Crear un modelo secuencial o funcional.
3. Compilar con optimizador, pérdida y métricas.
4. Entrenar con `fit()`.
5. Evaluar con `evaluate()`.
6. Predecir con `predict()`.

API secuencial: pila lineal de capas. **API funcional**: permite arquitecturas no secuenciales, múltiples entradas y múltiples salidas.

Ejemplo **Wide & Deep**: combina una ruta profunda para patrones complejos y una conexión directa para reglas simples.

Entradas, salidas múltiples y multitarea

Una red puede recibir varias entradas, por ejemplo datos tabulares y una imagen. También puede producir varias salidas: una principal y salidas auxiliares.

El aprendizaje multitarea puede mejorar generalización si las tareas comparten representaciones. Las salidas auxiliares también pueden actuar como regularización.

Ajuste de hiperparámetros

Hiperparámetros importantes:

- Número de capas ocultas.
- Número de neuronas por capa.
- Activaciones.
- Tasa de aprendizaje.
- Tamaño de lote.
- Optimizador.
- Regularización.

Se pueden usar GridSearchCV/RandomizedSearchCV con wrappers como SciKeras, Keras Tuner o métodos AutoML. La búsqueda evolutiva puede explorar arquitecturas e hiperparámetros.

Elección de arquitectura

Una capa oculta suficientemente grande puede aproximar funciones complejas, pero las redes profundas suelen ser más eficientes porque aprenden representaciones jerárquicas.

La entrada y salida vienen dadas por la tarea. En capas ocultas se puede usar el mismo número de neuronas por capa o diseños en pirámide. Una estrategia práctica es empezar con capacidad suficiente y regularizar, evitando cuellos de botella innecesarios.

Tasa de aprendizaje, lote e iteraciones

La tasa de aprendizaje suele ser el hiperparámetro más importante. Una forma práctica de buscarla es aumentarla progresivamente y escoger un valor algo menor que aquel donde la pérdida empieza a dispararse.

Lotes grandes aprovechan GPU, pero pueden generalizar peor o generar inestabilidad. Lotes pequeños introducen ruido útil, pero son menos eficientes.

El número de iteraciones suele controlarse con early stopping más que fijarse manualmente.

Problemas de entrenamiento en DNN

Además de gradientes desaparecidos/explosivos, hay retos como falta de datos etiquetados, coste de etiquetado, entrenamiento lento, millones de parámetros, ruido y sobreajuste.

Con sigmoides saturadas, las derivadas son casi cero y los gradientes no llegan a capas inferiores. Inicializaciones incorrectas pueden hacer crecer o disminuir la varianza capa a capa.

Inicialización con fan-in y fan-out

fan-in: número de entradas de una capa. **fan-out**: número de salidas.

La inicialización busca mantener varianzas de activaciones y gradientes estables:

- Glorot/Xavier usa un compromiso entre fan-in y fan-out.
- He/Kaiming es habitual con ReLU.

- LeCun es habitual con SELU.

Batch Normalization ampliada

BN calcula media y desviación del mini-batch, normaliza y aprende γ y β :

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad z = \gamma \hat{x} + \beta$$

Durante inferencia usa medias móviles acumuladas. Puede colocarse antes o después de activación. Añade coste, pero puede fusionarse con la capa anterior tras entrenar. Hiperparámetros: **momentum** y **axis**.

Gradient clipping

Recorta gradientes cuando superan un umbral. Es muy útil con RNN, donde los gradientes explosivos son frecuentes y Batch Normalization puede ser más difícil de aplicar.

Transfer learning, pretraining y autosupervisión

Transfer learning reutiliza capas inferiores porque suelen aprender patrones generales. Las capas superiores son más específicas y pueden sustituirse.

Procedimiento: congelar capas reutilizadas, entrenar cabeza nueva, descongelar algunas capas superiores y hacer fine-tuning con tasa menor.

Preentrenamiento no supervisado: usar muchos datos sin etiquetas para entrenar autoencoders o GANs y reutilizar partes del modelo.

Tarea auxiliar: entrenar en una tarea relacionada con etiquetas fáciles. Ejemplo: aprender si dos caras son de la misma persona antes de clasificar identidades.

Aprendizaje autosupervisado: generar etiquetas desde los datos, como enmascarar palabras y entrenar a predecirlas.

Optimizadores ampliados

Momentum: acumula velocidad. Parámetro típico $\beta = 0,9$.

Nesterov: calcula el gradiente mirando por delante de la posición actual.

AdaGrad: acumula gradientes cuadrados y reduce más la tasa en dimensiones con gradientes grandes. Puede frenarse demasiado pronto.

RMSProp: usa media exponencial de gradientes cuadrados recientes.

Adam: combina primer momento y segundo momento con corrección de sesgo inicial. Valores típicos: $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\eta = 0,001$.

AdaMax: variante de Adam con norma infinito. **Nadam**: Adam con Nesterov. **AdamW**: desacopla weight decay de Adam y suele generalizar mejor.

Métodos de segundo orden basados en Hessianos suelen ser impracticables en DNN por coste y memoria.

Programación de tasa de aprendizaje

Schedules frecuentes:

- Power scheduling.
- Exponential scheduling.
- Piecewise constant scheduling.
- Performance scheduling: reduce si validación no mejora.
- 1-cycle scheduling: sube y baja la tasa, puede producir superconvergencia.

Regularización avanzada

L2 restringe pesos grandes. L1 favorece modelos dispersos. La poda elimina pesos pequeños y puede apoyarse en herramientas como TensorFlow Model Optimization Toolkit.

Dropout muestrea una subred distinta en cada paso entre muchas posibles, compartiendo pesos; por eso se interpreta como un ensemble aproximado. La tasa p suele estar entre 10% y 50%, y normalmente no se aplica a la salida. Durante entrenamiento se escala por la probabilidad de mantener la neurona.

MC Dropout: mantiene dropout activo en test y promedia varias predicciones. Sirve para estimar incertidumbre y se relaciona con inferencia bayesiana aproximada.

Max-Norm: impone $\|w\|_2 \leq r$ y reescala pesos si superan el umbral. Reducir r aumenta regularización.

Recomendaciones prácticas

Normalizar entradas, reutilizar modelos preentrenados si existen, usar pretraining si hay pocos datos etiquetados, aplicar early stopping y elegir arquitectura suficiente pero regularizada. Para baja latencia: menos capas, activaciones rápidas, fusionar BN, usar dispersión o menor precisión. Para riesgo e incertidumbre: MC Dropout.

Preguntas tipo test del tema 5

1. Un perceptrón simple solo separa linealmente.
2. Un MLP puede resolver XOR gracias a capas ocultas y activaciones no lineales.
3. ReLU es típica en capas ocultas, softmax en multiclase excluyente.
4. Backprop usa la regla de la cadena.
5. Dropout se aplica normalmente en entrenamiento, no en test.
6. Adam combina ideas de momentum y RMSProp.
7. Gradiente desaparecido significa que las capas inferiores casi no aprenden.

Tema 6. Aprendizaje Profundo en Datos: CNN y Transformers

CNN

Una red convolucional contiene capas convolucionales y es especialmente útil en datos con estructura espacial, como imágenes.

Arquitectura típica:

Convolución $\rightarrow$ Activación $\rightarrow$ Pooling $\rightarrow$ Capas finales

Convolución

Un filtro o kernel se desplaza por la imagen, multiplica valores locales y suma para producir un mapa de activación.

Tamaño de salida:

$$Output = \frac{N + 2P - F}{S} + 1$$

donde N es tamaño de entrada, P padding, F tamaño del filtro y S stride.

Ejemplo: entrada $16 \times 16 \times 3$, 10 filtros 3×3 , padding 1, stride 1.

$$Output = \frac{16 + 2 \cdot 1 - 3}{1} + 1 = 16$$

Salida: $16 \times 16 \times 10$.

Parámetros por filtro:

$$3 \cdot 3 \cdot 3 + 1 = 28$$

Parámetros totales: $28 \cdot 10 = 280$.

Ventajas de CNN

- Conectividad local: cada neurona mira regiones pequeñas.
- Compartición de pesos: el mismo filtro se reutiliza en distintas posiciones.
- Menos parámetros que una capa densa.
- Equivarianza a traslación.

Trampa típica: Convolución es equivariante a traslación, no totalmente invariante. El pooling aporta cierta invariancia local.

Pooling

Reduce tamaño espacial y resume regiones.

- Max pooling: toma el máximo.
- Average pooling: toma la media.

Trampa típica: El pooling normalmente no aprende parámetros; la convolución sí.

Tareas de visión

- Clasificación: etiqueta global de imagen.
- Detección: localiza objetos con cajas y clases.
- Segmentación: etiqueta cada píxel o región.

Autoencoders

Arquitectura encoder-decoder que aprende una representación comprimida. Puede usarse para compresión, reducción de ruido o aprendizaje no supervisado de características.

Secuencias

Una secuencia tiene elementos, orden y contexto. Ejemplos: texto, audio, series temporales y vídeo. Primero se transforman elementos en vectores:

- Palabras o subpalabras $\rightarrow$ embeddings.
- Valores temporales $\rightarrow$ vectores de características.
- Imagen en Vision Transformers $\rightarrow$ parches convertidos en embeddings.

RNN y LSTM

Las RNN procesan paso a paso y mantienen un estado oculto. Las LSTM mejoran la memoria mediante puertas de olvido, entrada y salida. Aun así, procesan secuencialmente y tienen dificultad con dependencias muy largas.

Atención

La atención permite que cada token consulte directamente a otros tokens.

Queries, Keys y Values

A partir de embeddings se generan:

- Query Q : qué busco.
- Key K : qué ofrezco para ser encontrado.
- Value V : qué información transmito.

Atención escalada:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Trampa típica: Q y K deciden a quién mirar; V contiene la información que se recoge.

Self-attention, cross-attention y máscaras

Self-attention: Q , K y V salen de la misma secuencia.

Cross-attention: las queries salen de una secuencia y keys/values de otra. Típico en encoder-decoder.

Máscara causal: impide mirar tokens futuros en generación autoregresiva.

Transformer

Un bloque Transformer combina multi-head attention, feed-forward por token, conexiones residuales, layer normalization y codificación posicional.

- Encoder-only: comprensión y representaciones.
- Decoder-only: generación autoregresiva, como muchos LLMs.
- Encoder-decoder: traducción o transformación secuencia a secuencia.

Trampa típica: La atención por sí sola no conoce el orden; necesita codificación posicional.

Ventajas y limitaciones de Transformers

Ventajas:

- Capturan dependencias largas.
- Paralelizan mejor que RNN.
- Aprenden relaciones flexibles entre tokens.

Limitaciones:

- Coste cuadrático con el número de tokens.
- Mucho consumo de memoria.
- Necesitan muchos datos.
- Pueden aprender correlaciones espurias.

Representaciones jerárquicas en deep learning

Las redes profundas aprenden niveles de abstracción. En visión, las primeras capas pueden detectar bordes y texturas; capas intermedias detectan partes y formas; capas superiores detectan objetos o conceptos.

Esta jerarquía es una de las razones por las que deep learning reduce la necesidad de ingeniería manual de características.

Capa densa frente a capa convolucional

Una capa densa conecta cada entrada con cada neurona. En imágenes grandes esto dispara el número de parámetros y pierde estructura espacial.

Una capa convolucional usa filtros locales compartidos. Cada filtro se aplica en muchas posiciones, reduciendo parámetros y explotando localidad.

Convolución como operación local

Una convolución es una operación local lineal con una máscara o kernel. En procesamiento clásico de imagen, filtros como Gauss suavizan y Sobel detecta bordes. En CNN, los coeficientes de los filtros se aprenden con backpropagation.

Trampa típica: En CNN modernas no diseñamos manualmente filtros Sobel o Gauss para la tarea; la red aprende filtros útiles.

Historia: LeNet

LeCun mostró redes convolucionales tempranas para reconocimiento de dígitos manuscritos. LeNet-5 es un ejemplo clásico de CNN con convoluciones, pooling y capas finales para clasificación.

Detalles de convolución

Cada filtro tiene la misma profundidad que el volumen de entrada. Si la entrada es RGB con profundidad 3, un filtro 3×3 realmente tiene tamaño $3 \times 3 \times 3$.

Stride: salto del filtro. Stride mayor reduce resolución espacial.

Padding: bordes añadidos, normalmente ceros. Permite conservar tamaño espacial o evitar reducción demasiado rápida.

No toda combinación de tamaño de entrada, filtro, padding y stride es válida; la fórmula de salida debe dar entero.

Las convoluciones pueden aplicarse a entradas de tamaño variable, pero las capas densas finales suelen requerir tamaño fijo salvo que se usen estrategias como pooling global.

Activaciones, flatten y softmax en CNN

Las convoluciones se intercalan con activaciones no lineales, normalmente ReLU. Sin no linealidad, varias capas convolucionales equivaldrían a una transformación lineal.

Flatten: convierte mapas de características en un vector para capas densas finales.

En clasificación multiclase se usa softmax para obtener probabilidades por clase.

Problemas posibles: dying ReLU y vanishing gradients, ya vistos en fundamentos.

Machine Learning clásico frente a Deep Learning

En ML clásico suele haber dos fases: **feature engineering** manual y modelo final. En deep learning se aprende la representación dentro del modelo: **feature learning**. Esto permite entrenar de extremo a extremo, desde datos brutos hasta predicción.

Denoising autoencoders

Un autoencoder con eliminación de ruido recibe una entrada corrupta, por ejemplo con ruido o máscaras, y aprende a reconstruir la versión original. Obliga al modelo a capturar estructura útil de los datos.

Limitaciones de redes densas y CNN en secuencias

Una red densa trata la secuencia como bloque fijo: pierde flexibilidad ante longitudes variables y no modela bien orden temporal.

Una CNN puede capturar patrones locales en audio o series temporales, pero puede necesitar muchas capas para dependencias largas.

RNN y LSTM ampliadas

Una RNN aplica la misma celda en cada instante. El estado oculto viaja por la secuencia y resume información anterior.

Limitaciones:

- Cuello de botella: todo debe pasar por el estado oculto.
- Procesamiento secuencial, difícil de paralelizar.
- Dependencias largas difíciles, aunque LSTM lo mitiga.

Atención paso a paso

La atención reemplaza la memoria secuencial por comunicación directa entre tokens.

Pasos del scaled dot-product attention:

1. Calcular similitudes QK^T .

2. Escalar por $\sqrt{d_k}$ para estabilizar magnitudes.
3. Aplicar softmax para obtener pesos positivos que suman 1.
4. Combinar los valores V con esos pesos.

Multi-head attention

Multi-head attention aplica varias atenciones en paralelo. Cada cabeza puede aprender relaciones distintas: sintácticas, semánticas, locales o lejanas. Después se concatenan o combinan sus salidas.

Codificación posicional

La entrada de un Transformer es:

$$\text{embedding de contenido} + \text{encoding posicional}$$

La atención por sí sola es invariante al orden de los tokens; por eso necesita información posicional.

Bloque Transformer detallado

Componentes:

- Multi-head attention: mezcla información entre tokens.
- Feed-forward network: la misma red densa aplicada a cada token por separado.
- Conexiones residuales: suman entrada y salida del subbloque, estabilizando gradientes.
- Layer normalization: estabiliza escala de representaciones.
- Add & Norm: suma residual más normalización.

Encoder y decoder

Encoder: self-attention, Add & Norm, feed-forward, Add & Norm. Produce representaciones contextualizadas de toda la entrada.

Decoder: masked self-attention, cross-attention con la salida del encoder, feed-forward y normalizaciones. Genera paso a paso sin mirar tokens futuros.

Vision Transformers

Un Vision Transformer procesa imágenes como secuencias:

1. Divide la imagen en parches.
2. Convierte cada parche en embedding.
3. Añade codificación posicional.
4. Aplica bloques Transformer.
5. Usa la salida para clasificar, detectar o segmentar.

Transformers en series temporales

Aplicaciones: demanda eléctrica, sensores, tráfico, meteorología, finanzas y detección de anomalías.

Modelos mencionables:

- Informer.

- Autoformer.
- FEDformer.
- PatchTST.
- Chronos.
- Local Attention.

Modelos modernos y agentes

El Transformer original ha evolucionado con escala, entrenamiento con instrucciones, alineamiento con preferencias humanas, contextos largos, multimodalidad, uso de herramientas externas y agentes.

Ejemplos de familias actuales: ChatGPT, Gemini, Claude, Llama, Mistral, Qwen y DeepSeek.

Limitaciones adicionales

Además de coste cuadrático y memoria, los Transformers pueden ser difíciles de interpretar, necesitan posición para representar orden y pueden aprender correlaciones espurias si los datos tienen sesgos.

Preguntas tipo test del tema 6

1. En CNN los filtros se aprenden con backpropagation.
2. El filtro tiene la misma profundidad que la entrada.
3. Pooling reduce dimensiones y aporta cierta invariancia local.
4. Clasificación, detección y segmentación son tareas distintas.
5. Token no es embedding: el token es la unidad; el embedding es el vector.
6. Self-attention usa una secuencia; cross-attention conecta dos.
7. Un Transformer necesita información posicional.
8. Multi-head attention son varias atenciones en paralelo.

Tema 9. Aspectos Avanzados, Retos y Desafíos

Idea general

El tema avanza desde rendimiento hacia confiabilidad. Un modelo con alta accuracy puede no ser fiable si aprende atajos, sesgos, correlaciones espurias o falla fuera de distribución.

XAI: IA explicable

La XAI busca hacer comprensibles las decisiones de modelos complejos.

Motivos:

- Confianza en sistemas críticos.
- Depuración de modelos.
- Detección de sesgos, atajos y data leakage.
- Cumplimiento regulatorio y auditoría.

Ejemplo clásico: clasificador lobo/perro que aprende a mirar la nieve en vez del animal. Alta accuracy no implica que el modelo razone correctamente.

Taxonomía XAI

Criterio	Tipos
Momento	Pre-hoc: interpretable por diseño. Post-hoc: explicación posterior
Alcance	Local: explica una predicción. Global: explica comportamiento general
Dependencia	Agnóstico: cualquier modelo. Específico: requiere acceso interno

Métodos XAI

LIME: perturba localmente la entrada y ajusta un modelo simple interpretable. Agnóstico e intuitivo, pero puede ser inestable.

SHAP: usa valores de Shapley de teoría de juegos. Tiene fundamento axiomático, pero puede ser computacionalmente caro.

Contraejemplos: indican qué tendría que cambiar para obtener otra decisión. Son intuitivos, pero no siempre causales.

Integrated Gradients: integra gradientes desde una baseline hasta la entrada. Específico de redes.

LRP: propaga relevancia desde la salida hacia atrás capa por capa.

CRP: explica mediante conceptos, no solo píxeles.

Prototipos: modelos que razonan por similitud a ejemplos prototípicos.

Evaluación de explicaciones

Criterios habituales: fidelidad, robustez, localización, complejidad, concisión, prescriptividad y concordancia con el dominio.

Trampa típica: Una explicación bonita no garantiza que sea fiel al razonamiento real del modelo.

Trustworthy AI, Responsible AI y AI Safety

Trustworthy AI: IA lícita, ética y robusta.

Responsible AI: gobernanza, auditoría, responsabilidades y rendición de cuentas.

AI Safety: prevención de daños, accidentes y comportamientos peligrosos.

AI Security: protección frente a ataques e intrusiones.

Trampa típica: AI Safety y AI Security están relacionadas, pero no son lo mismo.

OOD y mundo abierto

Los modelos suelen entrenarse en mundo cerrado, donde test y entrenamiento comparten clases y distribución. El mundo real es abierto: nuevas clases, anomalías y cambios de distribución.

OOD Detection: detectar ejemplos fuera de distribución.

Trampa típica: Una predicción con alta confianza puede ser OOD; las redes pueden estar muy confiadas en entradas fuera de distribución.

Aprendizaje continuo

Continual Learning aprende tareas secuencialmente sin reentrenar desde cero y sin olvidar lo anterior.

Problema central: **catastrophic forgetting**, pérdida de rendimiento en tareas antiguas al aprender nuevas.

Escenarios:

- Task-incremental: se conoce el ID de tarea en test.
- Domain-incremental: misma tarea, cambia la distribución.
- Class-incremental: aparecen nuevas clases y el modelo debe distinguir todas.

EWC: protege pesos importantes usando la información de Fisher.

$$L_{EWC} = L_B + \lambda \sum_i F_i (\theta_i - \theta_{A,i}^*)^2$$

LoRA: congela el modelo base y aprende adaptadores de bajo rango. Reduce coste y es útil en modelos fundacionales.

Machine unlearning

Busca que un modelo olvide datos concretos sin reentrenar desde cero, comportándose como si nunca los hubiera visto.

Motivos: derecho al olvido, privacidad, extracción de datos, membership inference attacks y regulación.

Tipos:

- Olvido de muestra.
- Olvido de clase.
- Unlearning exacto.
- Unlearning aproximado.

Métodos:

- SISA: divide datos en shards y slices; reentrena solo lo afectado.
- SSD: reduce pesos importantes para datos a olvidar pero no para datos a retener.

Trampa típica: Unlearning no es borrar una fila de la base de datos; es eliminar su influencia en el modelo.

Causalidad y correlaciones espurias

Correlación: dos variables varían juntas.

Causalidad: intervenir en una variable cambia otra.

Confusor: variable oculta que causa ambas y genera correlación aparente.

Ejemplo: ventas de helado y ahogamientos aumentan con el calor. El helado no causa ahogamientos; el confusor es la temperatura.

Trampa típica: Correlación no implica causalidad.

Fairness y sesgos

Los modelos pueden aprender shortcut learning: usar fondos, objetos o atributos correlacionados pero no causales.

Ejemplos:

- Camello asociado a desierto, vaca asociada a prado.
- Modelo médico/no médico que usa fonendoscopio como atajo.

IA generativa y LLMs

Aparecen como contexto avanzado. Puntos clave:

- Explicar LLMs grandes es costoso.
- SHAP puede ser impracticable en modelos enormes.
- LoRA permite adaptar LLMs con pocos parámetros.
- En cadenas de agentes surge la pregunta de a qué agente explicar.

Regulación y gobernanza

El AI Act europeo y marcos de IA confiable refuerzan:

- Trazabilidad.
- Explicabilidad.
- Supervisión humana significativa.
- Documentación técnica.
- Gestión de riesgos.
- Auditoría y rendición de cuentas.

RED XAI y BLUE XAI

RED XAI se orienta a investigar, explorar y depurar modelos. Sirve para encontrar errores, atajos, sesgos y vulnerabilidades.

BLUE XAI se orienta a usuarios, confianza, ética, legalidad y comunicación de decisiones.

Trampa típica: XAI no es solo explicar al usuario final; también sirve para auditar y romper supuestos del modelo.

Taxonomía XAI con ejemplos

- Pre-hoc: árboles, reglas, modelos lineales, modelos basados en prototipos.
- Post-hoc: LIME, SHAP, Integrated Gradients, LRP, Grad-CAM.
- Local: una predicción concreta, contraejemplos, LIME local.
- Global: PDP, ICE, importancia global de variables.
- Agnóstico: LIME, SHAP, contraejemplos.
- Específico: Grad-CAM, Integrated Gradients, LRP, CRP.

SHAP, Integrated Gradients, LRP y CRP ampliados

SHAP se basa en valores de Shapley y satisface propiedades como eficiencia, simetría, linealidad y dummy. Su problema es el coste, que puede crecer exponencialmente con el número de variables.

Integrated Gradients integra gradientes desde una **baseline** hasta la entrada real. Tiene propiedades como sensibilidad e invariancia de implementación, pero depende de elegir bien la baseline.

LRP reparte la relevancia de la salida hacia atrás conservando relevancia total. Variantes: LRP- ϵ , LRP- γ y LRP- $\alpha\beta$. No necesita baseline.

CRP lleva la explicación de píxeles o features a conceptos semánticos. Busca no solo dónde mira el modelo, sino qué concepto usa para decidir.

Prototipos y explicación intrínseca

Modelos como ProtoPNet o ProtoPFormer razonan por similitud: “esto se parece a este prototipo”. La explicación forma parte del razonamiento del modelo, no es una aproximación externa post-hoc.

Evaluación y herramientas XAI

Quantus es un toolkit de métricas para explicaciones. Agrupa criterios como fidelidad, localización, complejidad y robustez.

REVEL propone evaluar explicaciones locales con fidelidad, concordancia, prescriptividad, concisión y robustez.

Propuestas XAI avanzadas

X-SHIELD: regulariza durante entrenamiento usando explicaciones o mapas de saliencia para que el modelo no use features espurias.

$$L_{total} = L_{cls} + \lambda L_{shield}$$

STOOD-X: combina detección OOD con explicaciones. Un ejemplo OOD debería estar alejado en espacio de características y la explicación debe ayudar a entender por qué.

Three-Level Framework: transforma explicaciones técnicas en explicaciones más comprensibles con ayuda de LLMs.

CUBIC: usa modelos visión-lenguaje para detectar conceptos sesgantes sin anotaciones previas.

Limitaciones abiertas de XAI

Problemas:

- No hay una métrica única consensuada.
- Las explicaciones pueden inducir confirmation bias.
- Explicar LLMs grandes puede ser prohibitivo.
- XAI multimodal es más compleja.
- En cadenas de agentes, no siempre está claro a qué componente pedir explicación.

Trustworthy AI basada en riesgo

La IA confiable usa un enfoque basado en riesgo: cuanto mayor impacto potencial, mayores requisitos de documentación, explicabilidad, robustez, supervisión humana y trazabilidad.

Para sistemas de alto riesgo, el AI Act exige obligaciones como trazabilidad de decisiones, explicabilidad, supervisión humana significativa y documentación técnica completa.

Robustez en el ciclo de vida

La robustez debe cubrir todo el ciclo de vida de IA: datos, entrenamiento, evaluación, despliegue y monitorización.

Amenazas:

- Perturbaciones y ruido.
- Entradas adversarias.
- Anomalías y datos OOD.
- Usos maliciosos.
- Backdoors o puertas traseras.

Mundo cerrado y mundo abierto

En mundo cerrado se asume que las clases y distribuciones futuras son las vistas durante entrenamiento. En mundo abierto aparecen nuevas clases, cambios de distribución y anomalías.

Tipos de OOD:

- ID: datos de la distribución esperada, por ejemplo CIFAR-10 si se entrenó con CIFAR-10.
- Near OOD: distribución parecida pero distinta, por ejemplo CIFAR-100.
- Far OOD: distribución muy diferente, por ejemplo EMNIST frente a CIFAR.

Aprendizaje continuo ampliado

Motivaciones: el mundo cambia, reentrenar desde cero es caro, puede haber restricciones de privacidad para conservar datos históricos y los humanos aprenden continuamente.

Catastrophic forgetting ocurre porque al optimizar una nueva tarea los gradientes modifican pesos importantes para tareas anteriores.

Regularización L2 simple penaliza todos los pesos por igual. EWC mejora esto penalizando más los pesos importantes según Fisher.

LoRA en continual learning congela el modelo base y aprende adaptadores:

$$W_{actualizado} = W_0 + BA$$

Limitaciones: puede requerir ID de tarea, y el número de adaptadores crece con tareas. Variantes: O-LoRA, LoRA-Pro y Adapter Merging.

Backward Transfer: mide cómo aprender tareas nuevas afecta a tareas antiguas. Si $BWT < 0$, hay olvido; si $BWT = 0$, no hay efecto; si $BWT > 0$, hay transferencia positiva.

Machine unlearning ampliado

Unlearning es casi el problema inverso al continual learning: CL añade conocimiento; unlearning elimina influencia específica.

Puede olvidar muestras, clases, todos los datos de un usuario o solo cierta información. El unlearning exacto busca equivalencia con reentrenar sin esos datos; el aproximado busca una solución práctica cercana.

Unlearning como CL puede usar una pérdida:

$$L(x) = L_{retain}(x) + L_{forget}(x)$$

Estrategias de olvido:

- Maximizar un logit incorrecto.
- Igualar logits para eliminar preferencia.
- Minimizar o alterar distancias en embeddings.

SISA: identifica shard y slice afectados, reentrena desde un checkpoint anterior y deja intactos los demás modelos.

SSD calcula importancia para datos a retener y a olvidar. Si un peso es importante para olvidar pero no para retener, se amortigua. Limitaciones: coste de Fisher, calibración de hiperparámetros y posible caída de rendimiento.

Problema abierto: verificar que un dato fue realmente olvidado y que es indistinguible de un dato nunca visto.

Causal Discovery

Cuando no se puede intervenir, la **causal discovery** intenta inferir estructura causal desde datos observados usando dependencias e independencias condicionales.

Requisitos fuertes:

- Suficiencia causal: observar confusores relevantes.
- DAG: asumir un grafo dirigido acíclico.

Causal Representation Learning

Busca aprender representaciones latentes que separen factores causales de factores espurios.

Se suele pensar en una variable latente Z que genera observaciones $x = f(Z)$. En imágenes, Z puede contener contenido, estilo, fondo e iluminación.

Descomposición:

- Z_C : contenido causal o invariante, por ejemplo la vaca.
- Z_S : estilo o fondo espurio, por ejemplo playa o prado.

Autoencoders pueden representar este proceso mediante encoder, espacio latente y decoder.

La supervisión débil o intervenciones de estilo buscan cambiar Z_S manteniendo Z_C estable.

V-REx

Variance Risk Extrapolation busca representaciones invariantes entre entornos. Minimiza riesgo medio y varianza del riesgo:

$$R_{V-REx} = \sum_e R_e + \beta \text{Var}(\{R_e\}_e)$$

Si una característica funciona solo en un entorno, aumenta la varianza del riesgo. Si es causal/invariante, debería funcionar en varios entornos.

Preguntas tipo test del tema 9

1. Alta accuracy no implica modelo confiable.
2. LIME y SHAP son métodos post-hoc agnósticos.
3. Pre-hoc significa interpretable por diseño.
4. Local explica una predicción; global explica el comportamiento general.
5. OOD no es necesariamente adversarial.
6. Catastrophic forgetting ocurre en aprendizaje continuo.
7. EWC protege pesos importantes, no todos por igual.
8. Machine unlearning busca eliminar influencia de datos usados.
9. Contraejemplo no implica causalidad real.
10. Trustworthy AI incluye licitud, ética y robustez.

Chuleta Final de Examen Tipo Test

Diferencias que suelen caer

- Clasificación predice clases; regresión predice valores.
- Clustering descubre grupos; clasificación asigna clases conocidas.
- Accuracy mide aciertos totales; precision mira positivos predichos; recall mira positivos reales.
- ROC usa TPR frente a FPR ; PR suele ser mejor en datasets desbalanceados.
- Ridge es L2; Lasso es L1 y puede poner pesos a cero.
- Bagging reduce varianza y paraleliza; boosting es secuencial y corrige errores.
- PCA preserva varianza; LDA busca separar clases.
- Dropout se aplica en entrenamiento; en test normalmente se desactiva.
- Convolución es equivariante; pooling aporta cierta invariancia.
- Self-attention usa una secuencia; cross-attention conecta dos.
- XAI post-hoc explica después; pre-hoc interpreta desde el diseño.
- Correlación no implica causalidad.

Fórmulas mínimas

$$\hat{y} = \theta^T x$$

$$RMSE = \sqrt{\frac{1}{m} \sum (\hat{y} - y)^2}, \quad MAE = \frac{1}{m} \sum |\hat{y} - y|$$

$$\sigma(t) = \frac{1}{1 + e^{-t}}, \quad p_k = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

$$silueta = \frac{b - a}{\max(a, b)}$$

$$Output = \frac{N + 2P - F}{S} + 1$$

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Si solo puedes memorizar 25 ideas

1. Supervisado implica etiquetas.
2. No supervisado no implica que no haya entrenamiento; implica ausencia de etiquetas.
3. Generalización es rendimiento en datos no vistos.
4. No uses test para ajustar hiperparámetros.
5. Sobreajuste: train bien, test mal.
6. Regularización simplifica o restringe el modelo.
7. En clases desbalanceadas, accuracy engaña.
8. Precisión y recall se mueven con el umbral.

9. kNN depende de escala y distancia.
10. SVM busca máximo margen.
11. Árboles interpretables sobreajustan fácilmente.
12. Random Forest reduce varianza de árboles.
13. Boosting corrige errores secuencialmente.
14. Datos malos limitan el modelo.
15. Outlier puede ser dato válido.
16. Min-max sensible a extremos; z-score usa media y desviación.
17. K-Means necesita k y falla con densidades/formas raras.
18. DBSCAN detecta ruido y no necesita k .
19. PCA no usa etiquetas.
20. Perceptrón simple no resuelve XOR.
21. Backprop usa gradientes y regla de la cadena.
22. Dropout regulariza apagando neuronas en entrenamiento.
23. CNN aprende filtros locales compartidos.
24. Transformers necesitan posición y tienen coste cuadrático en tokens.
25. Alta accuracy no garantiza fiabilidad, causalidad ni ausencia de sesgos.